

rag-chat-node

API REST de Chat RAG (Retrieval-Augmented Generation) com Claude

Documento de Engenharia de Software

Levantamento de Requisitos, Casos de Uso, Arquitetura e Plano de Testes

Projeto de portfólio pessoal

Autor: Bruno Silveira

15 de julho de 2026

Sumário

1	Introdução	2
1.1	Contexto e objetivo	2
1.2	Escopo	2
1.3	Tecnologias utilizadas	2
2	Levantamento de Requisitos	2
2.1	Atores do sistema	3
2.2	Requisitos funcionais	3
2.3	Requisitos não funcionais	3
2.4	Regras de negócio	4
3	Casos de Uso	4
3.1	Diagrama de casos de uso	5
3.2	Especificação dos casos de uso	5
3.2.1	UC01 – Ingerir Documento	5
3.2.2	UC02 – Listar Documentos	5
3.2.3	UC03 – Fazer Pergunta (Chat RAG)	5
3.2.4	UC04 – Consultar Histórico	6
3.2.5	UC05 – Verificar Disponibilidade da API	6
4	Arquitetura do Sistema	6
4.1	Visão arquitetural em camadas	6
4.2	Estrutura de diretórios	7
4.3	Decisões de arquitetura (ADRs)	7
4.4	Diagrama de implantação (Docker)	8
5	Plano de Testes	8
5.1	Estratégia de testes	8
5.2	Casos de teste	9
5.3	Evidência de execução	9
6	Como Executar o Projeto	10
7	Conclusão	10

1 Introdução

1.1 Contexto e objetivo

O **rag-chat-node** é uma API REST desenvolvida como projeto de portfólio para demonstrar, na prática, a construção de um pipeline de *Retrieval-Augmented Generation* (RAG) sobre Node.js/TypeScript: ingestão de documentos textuais, divisão em pedaços (*chunking*), geração de *embeddings*, recuperação por similaridade semântica e geração de resposta em linguagem natural com um modelo de linguagem (Claude, da Anthropic).

O objetivo deste documento é registrar, de forma formal, o processo de engenharia de software aplicado ao projeto: o levantamento de requisitos, a especificação dos casos de uso, as decisões arquiteturais tomadas (registradas como ADRs – *Architecture Decision Records*) e o plano de testes utilizado para validar o sistema.

1.2 Escopo

O escopo da API compreende:

- Ingestão de documentos textuais, com chunking automático e geração de embeddings locais (sem custo de API externa);
- Listagem dos documentos já ingeridos;
- Perguntas e respostas em linguagem natural, fundamentadas no conteúdo dos documentos ingeridos (RAG), com o Claude gerando a resposta final;
- Motor de resposta heurístico de fallback, para uso sem chave de API paga;
- Histórico consultável de perguntas e respostas;
- Documentação interativa da própria API via OpenAPI/Swagger UI.

Está fora do escopo desta versão: autenticação/autorização de usuários, persistência em banco de dados (armazenamento é em memória) e upload de arquivos binários (PDF, DOCX) – apenas texto puro é aceito.

1.3 Tecnologias utilizadas

Camada	Tecnologia
Runtime	Node.js 20 (LTS)
Linguagem	TypeScript (strict)
Framework HTTP	Express
Validação	Zod
LLM	Anthropic Claude API (@anthropic-ai/sdk)
Embeddings	Modelo local via @xenova/transformers (all-MiniLM-L6-v2)
Vector store	Em memória (array + similaridade de cosseno)
Documentação de API	OpenAPI 3.0 + Swagger UI
Testes	Vitest + Supertest
Containerização	Docker + Docker Compose
Versionamento	Git (Git Flow) + Conventional Commits

2 Levantamento de Requisitos

2.1 Atores do sistema

Ator	Descrição
Usuário da API	Sistema cliente (front-end, script, Postman/Insomnia) que consome a API REST via HTTP. Não há diferenciação de perfis nesta versão.
Claude (Anthropic API)	Ator externo responsável por gerar a resposta em linguagem natural a partir da pergunta e do contexto recuperado.
Modelo de Embeddings Local	Ator externo, executado dentro do próprio processo Node.js via <code>@xenova/transformers</code> , responsável por converter texto em vetores numéricos.

2.2 Requisitos funcionais

ID	Descrição	Prioridade
RF01	O sistema deve permitir a ingestão de um documento textual (<code>title + text</code>) via <code>POST /documents</code> .	Alta
RF02	O sistema deve dividir o texto em <i>chunks</i> de tamanho configurável, com sobreposição entre chunks consecutivos.	Alta
RF03	O sistema deve gerar um embedding vetorial para cada chunk, usando um modelo local (sem custo de API externa).	Alta
RF04	O sistema deve armazenar os chunks embedded de forma buscável por similaridade.	Alta
RF05	O sistema deve permitir listar os documentos ingeridos via <code>GET /documents</code> .	Média
RF06	O sistema deve permitir enviar uma pergunta via <code>POST /chat</code> .	Alta
RF07	O sistema deve recuperar os N chunks mais similares à pergunta (cosseno), com N configurável (<code>RETRIEVAL_TOP_K</code>).	Alta
RF08	O sistema deve montar um prompt com os chunks recuperados como contexto e enviá-lo ao Claude.	Alta
RF09	Sem <code>ANTHROPIC_API_KEY</code> configurada, o sistema deve gerar uma resposta heurística determinística, com o mesmo contrato de saída.	Alta
RF10	O sistema deve registrar cada par pergunta/resposta em um histórico.	Média
RF11	O sistema deve permitir consultar o histórico via <code>GET /history</code> , mais recentes primeiro.	Média
RF12	O histórico deve ser limitado a <code>HISTORY_MAX_ITEMS</code> itens.	Baixa
RF13	O sistema deve expor <code>GET /health</code> para verificação de disponibilidade.	Média
RF14	O sistema deve expor documentação interativa (Swagger UI) em <code>GET /docs</code> .	Média

2.3 Requisitos não funcionais

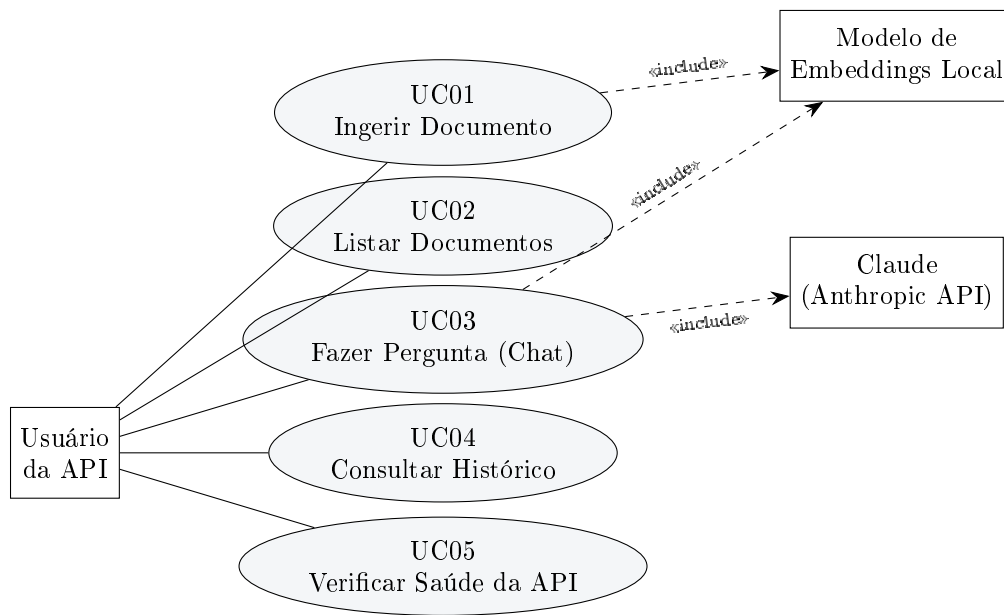
ID	Descrição	Categoria
RNF01	Requisições inválidas devem retornar 400 Bad Request com detalhes do erro de validação.	Confiabilidade
RNF02	Erros inesperados devem ser tratados por um middleware central, retornando 500 , sem derrubar o processo.	Confiabilidade
RNF03	O projeto deve subir com um único comando (docker compose up -build).	Portabilidade
RNF04	O pipeline de embeddings não deve exigir chave de API paga.	Custo
RNF05	A API deve ser implementada em TypeScript com checagem estática (strict: true).	Manutenibilidade
RNF06	Deve haver cobertura de testes automatizados nos módulos determinísticos e testes de integração dos endpoints HTTP.	Testabilidade
RNF07	Os endpoints devem ser documentados via OpenAPI 3.0.	Usabilidade
RNF08	O corpo aceito em requisições deve ser limitado a 5 MB.	Segurança

2.4 Regras de negócio

ID	Descrição
RN01	Um documento vazio (texto em branco após normalização) não pode ser ingerido – a ingestão falha de forma explícita.
RN02	O overlap entre chunks deve ser sempre menor que o tamanho do chunk (chunkOverlap < chunkSize).
RN03	A resposta de /chat deve sempre ter o mesmo formato, com ou sem o Claude configurado – o consumidor da API não trata dois contratos diferentes.
RN04	Perguntas sem nenhum documento ingerido ainda devem ser respondidas (contexto vazio), não devem retornar erro.

3 Casos de Uso

3.1 Diagrama de casos de uso



3.2 Especificação dos casos de uso

3.2.1 UC01 – Ingerir Documento

Ator principal: Usuário da API **Ator secundário:** Modelo de Embeddings Local

Pré-condições: Nenhuma.

Fluxo principal:

1. O usuário envia `POST /documents` com `title` e `text`;
2. O sistema valida o corpo da requisição (RNF01);
3. O sistema divide o texto em chunks com overlap (RF02);
4. Para cada chunk, o sistema gera o embedding via modelo local (RF03);
5. O sistema armazena os chunks embedded no vector store (RF04);
6. O sistema registra os metadados do documento e retorna `201 Created`.

Fluxos alternativos: (A1) corpo inválido/texto vazio → `400 (RN01)`; (A2) falha no modelo de embeddings → `500`, nenhum dado parcial é persistido.

Pós-condições: Documento e seus chunks/embeddings persistidos no estado em memória da aplicação.

3.2.2 UC02 – Listar Documentos

Ator principal: Usuário da API

Fluxo principal: `GET /documents` → lista ordenada por data de criação, com `id`, `title`, `createdAt`, `chunkCount` (RF05).

Fluxo alternativo: Nenhum documento ingerido → `200` com lista vazia.

Pós-condições: Nenhuma (operação de leitura).

3.2.3 UC03 – Fazer Pergunta (Chat RAG)

Ator principal: Usuário da API **Atores secundários:** Modelo de Embeddings Local, Claude API (ou motor heurístico)

Pré-condições: Nenhuma (funciona mesmo sem documentos ingeridos, RN04).

Fluxo principal:

1. O usuário envia `POST /chat` com `question`;
2. O sistema valida o corpo da requisição (RNF01);
3. O sistema gera o embedding da pergunta (RF07);
4. O sistema busca os `RETRIEVAL_TOP_K` chunks mais similares (RF07);
5. O sistema monta o prompt com o contexto recuperado (RF08);
6. Com Claude configurado, chama a API do Claude; sem chave, gera resposta heurística a partir do chunk mais similar (RF09);
7. O sistema registra pergunta, resposta e ids dos chunks no histórico (RF10);
8. O sistema retorna `200 OK` com o item de histórico criado.

Fluxos alternativos: (A1) `question` vazia → `400`; (A2) sem chave do Claude → resposta heurística, mesmo contrato de saída (RN03); (A3) nenhum documento ingerido → contexto vazio, fluxo continua (RN04); (A4) falha na chamada ao Claude → `500`, nada é adicionado ao histórico.

Pós-condições: Novo item adicionado ao histórico.

3.2.4 UC04 – Consultar Histórico

Ator principal: Usuário da API

Fluxo principal: `GET /history` → lista de itens, mais recentes primeiro, limitada a `HISTORY_MAX_ITEMS` (RF11, RF12).

Fluxo alternativo: Nenhuma pergunta feita ainda → `200` com lista vazia.

Pós-condições: Nenhuma (operação de leitura).

3.2.5 UC05 – Verificar Disponibilidade da API

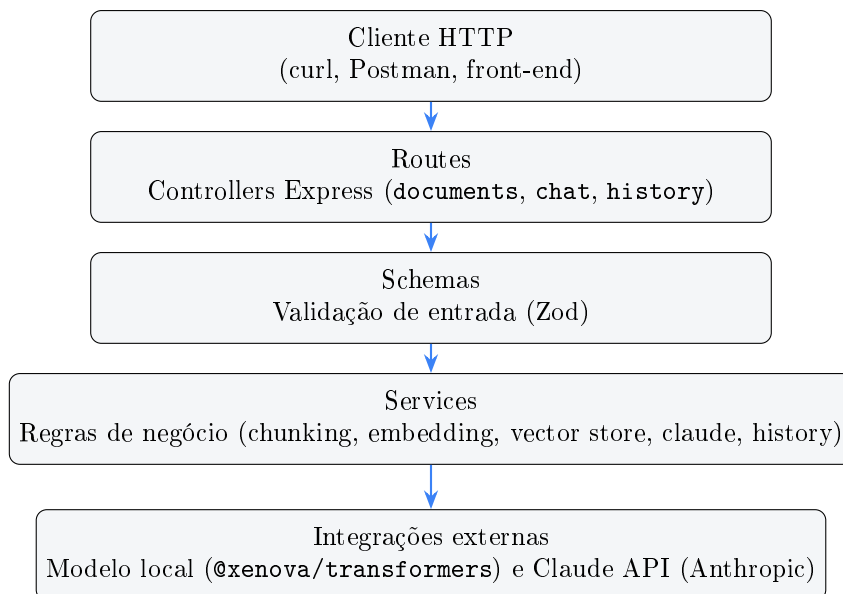
Ator principal: Usuário da API (tipicamente uma ferramenta de monitoramento)

Fluxo principal: `GET /health` → `200 OK` com `{"status": "ok"}`. Não depende de nenhum recurso externo.

4 Arquitetura do Sistema

4.1 Visão arquitetural em camadas

A aplicação segue uma arquitetura em camadas (*layered architecture*), isolando responsabilidades HTTP, validação, regra de negócio e integrações externas.



Erros de qualquer camada são propagados via `next(err)` até um middleware único (`error-handler.ts`), que decide o status HTTP e o formato da resposta de erro – ver ADR-04.

4.2 Estrutura de diretórios

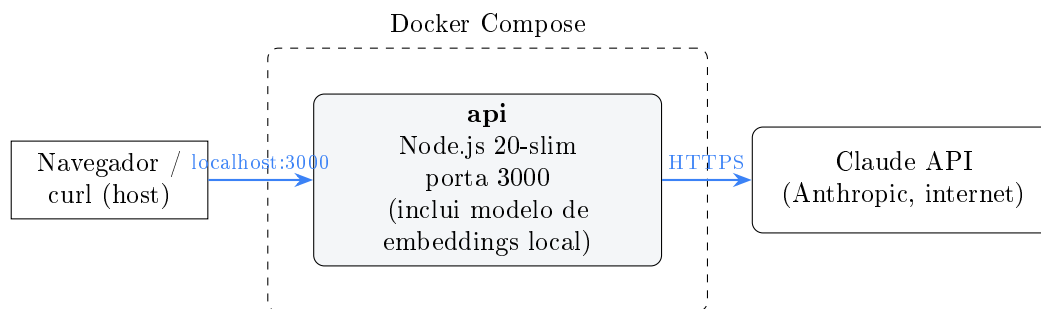
```
rag-chat-node/  
- src/  
  - server.ts bootstrap do Express + Swagger UI  
  - config.ts leitura de variáveis de ambiente  
  - routes/ documents.route.ts, chat.route.ts, history.route.ts  
  - schemas/ documents.schema.ts, chat.schema.ts (Zod)  
  - services/ chunking, embedding, vectorStore, document,  
               claude, history, chat  
  - middlewares/ error-handler.ts  
  - utils/ cosineSimilarity.ts  
  - types/ index.ts  
- tests/ 6 arquivos, 23 testes (unitários + integração)  
- openapi.yaml especificação OpenAPI 3.0  
- Dockerfile, docker-compose.yml
```

4.3 Decisões de arquitetura (ADRs)

Decisão	Contexto e justificativa
ADR-01: Vector store em memória (Repository Pattern)	Escopo de portfólio, não produção com milhões de documentos. Um banco vetorial externo (Pinecone, Qdrant) adicionaria infraestrutura sem necessidade real neste volume. <code>InMemoryVectorStore</code> expõe uma interface pequena (<code>add/search/clear</code>) que isola o resto da aplicação da estrutura de armazenamento – trocar por um banco vetorial futuramente significa reimplementar apenas essa classe.

Decisão	Contexto e justificativa
ADR-02: Motor duplo (Claude + heurístico)	Uma chave de API paga não deve ser requisito para demonstrar/testar o projeto. <code>claude.service.ts</code> verifica se <code>ANTHROPIC_API_KEY</code> está presente; sem ela, cai em uma resposta heurística determinística, com o mesmo contrato de saída (RN03). O projeto fica 100% demonstrável sem custo e passa a usar IA real assim que a chave é configurada, sem alteração de código.
ADR-03: Embeddings locais (Lazy Singleton)	Evita uma segunda dependência de API paga só para vetorização. O modelo (Xenova/all-MiniLM-L6-v2) roda no próprio processo Node.js via ONNX runtime; o pipeline é carregado uma única vez (Promise armazenada em módulo), evitando múltiplas inicializações concorrentes.
ADR-04: Validação centralizada (Zod + middleware único)	Cada rota declara um schema Zod e delega erros de validação/negócio a um único <code>error-handler</code> , evitando duplicação de <code>try/catch</code> e garantindo contrato de erro consistente entre endpoints.
ADR-05: Sem autenticação no MVP	Escopo do projeto é demonstrar o pipeline RAG, não um sistema multiusuário. Todos os endpoints são públicos nesta versão – restrição documentada explicitamente, não um esquecimento; autenticação por API key seria adicionada como middleware Express, sem alterar a camada de serviços.

4.4 Diagrama de implantação (Docker)



A imagem é construída em duas etapas (*multi-stage build*): a etapa **build** instala todas as dependências e compila o TypeScript (`tsc`); a etapa **runtime** instala apenas dependências de produção e copia os artefatos compilados, mantendo a imagem final enxuta. A base é `node:20-slim` (Debian/glibc) em vez de Alpine, pois os binários nativos do `onnxruntime-node` (usado para embeddings) dependem de glibc e não funcionam sobre musl. Um volume nomeado persiste o cache do modelo de embeddings entre reinícios do container, evitando redownload.

5 Plano de Testes

5.1 Estratégia de testes

A estratégia combina dois níveis, ambos com Vitest:

- **Testes unitários:** módulos determinísticos e puros, sem I/O (`chunking.service`, `cosineSimilarity`, `vectorStore.service`) – rodam em milissegundos;

- **Testes de integração** (Supertest): endpoints HTTP completos sobre a **app** do Express, exercitando o pipeline real de ponta a ponta (chunking → embedding local real → vector store → resposta heurística, já que os testes não configuram **ANTHROPIC_API_KEY**).

Não há mocks do modelo de embeddings: os testes de integração usam o modelo real **Xenova/all-MiniLM-L6-v2**, validando o pipeline de ponta a ponta.

5.2 Casos de teste

Arquivo de teste	Cenários cobertos	Tipo
<code>chunking.test.ts</code>	Texto vazio, texto menor que o chunk, divisão com overlap, unicidade de ids, erros de parâmetro inválido (6 casos).	Unitário
<code>cosineSimilarity.test.ts</code>	Vetores idênticos, ortogonais, opostos, com norma zero, e de dimensões diferentes (5 casos).	Unitário
<code>vectorStore.test.ts</code>	Armazenar/contar, buscar top-K ordenado por score, remover por documento, limpar (4 casos).	Unitário
<code>documents.route.test.ts</code>	POST /documents ingere e retorna metadados; 400 para texto vazio; GET /documents lista (3 casos).	Integração
<code>chat.route.test.ts</code>	POST /chat responde em modo heurístico; 400 para pergunta vazia; responde mesmo sem documentos ingeridos (3 casos).	Integração
<code>history.route.test.ts</code>	GET /history vazio inicialmente; ordem mais recente primeiro após perguntas (2 casos).	Integração

5.3 Evidência de execução

Suíte executada localmente com **npx vitest run**:

```

RUN v2.1.9 rag-chat-node

(pass) tests/chunking.test.ts (6 tests)
(pass) tests/vectorStore.test.ts (4 tests)
(pass) tests/cosineSimilarity.test.ts (5 tests)
(pass) tests/documents.route.test.ts (3 tests)
(pass) tests/chat.route.test.ts (3 tests)
(pass) tests/history.route.test.ts (2 tests)

Test Files 6 passed (6)
Tests 23 passed (23)

```

Além da suíte automatizada, o fluxo completo foi validado manualmente contra a imagem Docker real (**docker build + docker run**, base **node:20-slim**): **GET /health** (200), **POST /documents** ingerindo um documento de teste (201, **chunkCount: 1**), **POST /chat** respondendo em modo heurístico com o trecho correto recuperado (200), **GET /history** refletindo a interação (200)

e `GET /docs` servindo o Swagger UI (200) – confirmando que o pipeline de embeddings local funciona corretamente dentro do container, não apenas em ambiente de desenvolvimento local.

6 Como Executar o Projeto

```
cd rag-chat-node
cp .env.example .env # opcional: preencher ANTHROPIC_API_KEY para respostas reais
docker compose up --build
```

A API fica disponível em <http://localhost:3000>, com documentação interativa em <http://localhost:3000/docs>.

Para rodar a suíte de testes automatizados:

```
cd rag-chat-node
npm install
npm test
```

Exemplos de uso via `curl` estão documentados em [docs/05-documentacao-api.md](#).

7 Conclusão

Este documento formalizou o processo de engenharia de software aplicado ao rag-chat-node, desde o levantamento de requisitos até a arquitetura implementada e o plano de testes que a valida. O projeto foi desenvolvido como peça de portfólio, com foco em demonstrar boas práticas replicáveis em contexto profissional: separação clara de camadas, regras de negócio testáveis e isoladas, um motor duplo que mantém a aplicação 100% demonstrável sem custo, documentação de API via OpenAPI, versionamento seguindo Git Flow com commits convencionais, e empacotamento reprodutível via Docker.